

Tomato Leaf Disease Detection

AI-Based Crop Disease Diagnosis: From Research Model to Mobile Application

Submitted by	
Habiba	0272130005101046
Mithila Akter	0272130005101048
Imran Hossain Badsha	0272130005101012
Alfe Sani	1935202085

Supervised By

Faria Rahman Annasha, Lecturer
Department of CSE
City University of Bangladesh

Table of Contents

1. Introduction
2. Background Study
3. Problem Statement
4. Objectives
5. Literature Review
6. System Overview
7. Methodology — Model Research
8. Methodology — Mobile Application
9. Technology Stack
10. System Design & Architecture (Mobile App)
11. Implementation Details
12. Results and Evaluation
13. Working Schedule
14. Limitations
15. Expected Outcome and Future Work
16. Conclusion
17. References

1. Introduction

Tomato is one of the most widely grown vegetable crops, but its yield is often damaged by leaf diseases such as Early Blight, Bacterial Spot, and Leaf Miner infestation. Farmers usually detect these diseases by eye, which is slow and often wrong, especially for farmers who are not trained in plant pathology.

This project builds a complete, two-part solution for this problem:

1. **A deep learning model (YOLOv12n)** trained to detect and classify tomato leaf diseases from images.
2. **A mobile application ("Plant Health")** built in Flutter that lets any farmer take or upload a photo of a leaf and instantly get a diagnosis and treatment suggestion, currently powered by Google's Gemini vision AI, with the trained YOLO model planned as an on-device upgrade.

Together, these two parts move the project from a research idea to something a real farmer could actually use in the field.

2. Background Study

Plant disease is a major cause of crop loss around the world. In Bangladesh, tomato farming is common but many farmers do not have easy access to agricultural experts. A wrong or late diagnosis can lead to the wrong pesticide being used, wasted money, and lower harvest.

At the same time, smartphones are now common even in rural areas, and recent progress in computer vision (YOLO family of models) and cloud AI (like Google Gemini) makes it possible to build a tool that gives instant, accurate answers from a simple photo — no expert needed on site.

3. Problem Statement

- Manual disease detection by farmers is slow, inconsistent, and error-prone.
- Expert agricultural help is not always available, especially in rural areas.
- Existing research models (CNN/YOLO based) are usually tested only in a lab/notebook environment and are rarely turned into something a farmer can use on a phone.
- There is a need for a system that is both **accurate** (research-grade detection) and **usable** (a simple mobile app that works in real conditions, including patchy internet and possible service outages).

4. Objectives

- To implement a YOLOv12n-based model that detects and classifies tomato leaf diseases in near real time.
- To collect and annotate a labeled dataset of tomato leaf images for training and evaluation.
- To build a cross-platform mobile application (Android/iOS) that any farmer can use to photograph a leaf and get an instant, understandable result in Bengali.
- To make the mobile application reliable in real-world conditions — handling network failures, server downtime, and API key expiry gracefully instead of crashing.
- To compare the performance of the developed model against existing published work.

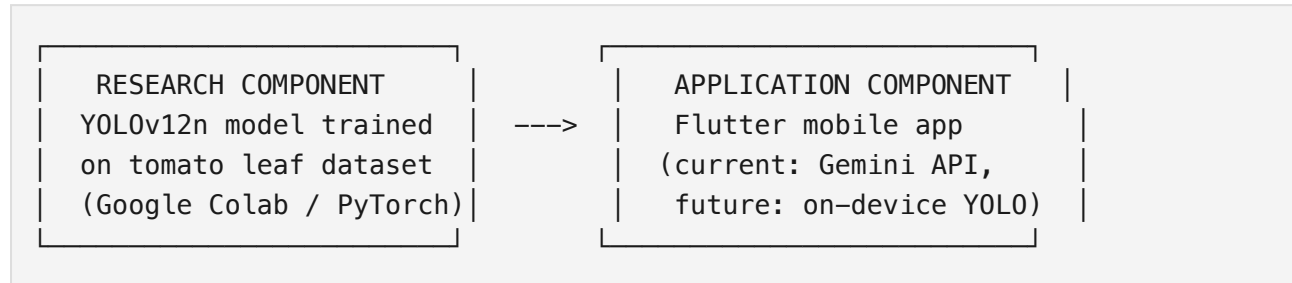
5. Literature Review

Author(s)	Year	Model/Approach	Achievements	Gaps/Limitations
Rajamohanam, R., & Latha, B. C.	2023	Optimized YOLOv5	93% accuracy in tomato leaf disease classification	Background interference, limited to two disease categories
Abulizi, A., Ye, J., & Guo, W.	2024	DM-YOLO (YOLOv9)	Precision increased by 2–3% vs. improved models	Small lesions and overlapping symptoms remain challenging
Tm, P., Pranathi, A., & Sai Ashritha, K.	2018	LeNet CNN	94–95% accuracy with minimal computing resources	Limited to simpler CNN architectures
Shanthi, D. L., Vinutha, K., & Ashwini, N.	2024	Custom CNN (AlexNet, VGGNet-16)	Improved classification accuracy	High computational demand for large datasets
Altalak, M., Uddin, M. A., Alajmi, A., & Rizg, A.	2022	Hybrid CNN, SVM, CBAM	97.2% accuracy in disease detection	Needs improvement in the learning process
Agarwal, M., Gupta, S. K., & Biswas, K. K.	2020	Simplified CNN (8 hidden layers)	98.4% accuracy on PlantVillage dataset	Needs more image-preprocessing optimization

Summary: Most existing work reports high accuracy but is trained/tested only on lab datasets and simple, uncrowded images. Very few studies discuss turning the model into a usable, real-time farmer-facing product — this is the gap our project addresses.

6. System Overview

The project has two connected halves:



- The **research component** proves the disease-detection concept works and measures its accuracy scientifically.
- The **application component** is the working, installable proof that the idea can reach real farmers today, using a cloud AI (Gemini) as the detection engine while the custom model is finalized for on-device use.

7. Methodology — Model Research

7.1 Dataset

- **Source:** PlantVillage / Tomato-Village Dataset
- **Classes:** Leaf Miner, Bacterial Spot, Early Blight, Healthy
- **Total images:** 794 annotated images
- **Split:** 697 training / 77 validation / 20 test images

7.2 Data Annotation

YOLO-family models need labeled "ground truth" — bounding boxes drawn around each diseased area. This was done using **Make Sense AI**, a web-based annotation tool, which exports each image's labels as a `.txt` file containing class IDs and box coordinates.

7.3 Training Environment

Item	Detail
Platform	Google Colab (cloud-based Python environment)
GPU	NVIDIA Tesla T4 Tensor Core, ~15 GB VRAM
CUDA	12.4 (driver 550.54.15)
System RAM	~12.7 GB
Language	Python 3.12.12
Framework	PyTorch 2.9.0 + CUDA 12.6
Model orchestration	Ultralytics 8.3.63
Model	YOLOv12n (nano variant, optimized for edge/mobile devices)

7.4 Training Process

1. The training set (697 images) is fed to the model along with its annotations.
2. YOLOv12n breaks each image into patches and learns visual patterns (leaf texture, colored spots, edge deformities).
3. These patterns are converted into numeric (binary) data the model can process mathematically.
4. The model repeatedly compares its predictions to the human annotations and adjusts itself to reduce error (loss function), over 100 training epochs.

8. Methodology — Mobile Application

The mobile app follows a simple, repeatable flow:

```
Open app → Login/Signup (Firebase Auth) → Home
  → Take photo / Choose from gallery
  → Preview image → Tap "Analyze"
  → Image sent to AI for analysis
  → Result shown: healthy / disease name, severity, symptoms, and suggested
  medicine
```

Development approach:

- Built using **Flutter**, so the same codebase runs on both Android and iOS.
- Follows **Clean Architecture** (Presentation → Domain → Data layers) so the AI backend can be swapped (e.g., from Gemini to the trained YOLO model) without rewriting the UI.
- Uses the **BLoC pattern** to manage app state (loading, success, error) in a predictable way.

9. Technology Stack

Research / Model Training

- Python, PyTorch, Ultralytics (YOLOv12n)
- Google Colab (NVIDIA Tesla T4 GPU)
- Make Sense AI (annotation tool)

Mobile Application

- Flutter & Dart
- BLoC (`flutter_bloc`) for state management
- GetIt for dependency injection
- Firebase Authentication (login/signup/profile)
- Firebase Remote Config (manages AI API keys remotely, without releasing a new app version)
- Google Gemini API (`gemini-flash-latest`) for image-based disease analysis
- HTTP package for networking
- Image Picker (camera & gallery access)
- Open-Meteo API for weather information

10. System Design & Architecture (Mobile App)

The app is organized in four layers, so each part has one clear responsibility:

```
UI (Pages/Widgets)
  ↓ user action
BLoC (Events → States)
  ↓
UseCase (business rule: "analyze this image")
  ↓
Repository (defines the contract)
  ↓
DataSource (talks to the Gemini API over HTTPS)
```

10.1 Reliable API Key Management

Because a single AI API key can expire, get rate-limited, or be revoked, the app supports a **pool of keys** instead of one:

- Keys are supplied through Firebase Remote Config (so they can be rotated without publishing a new app update) or through a build-time configuration for development.
- If a request fails because a key is expired, blocked, or out of quota, the app automatically retries with the next key in the pool — invisible to the user.
- If the AI server itself is temporarily overloaded (a server-side error, not a key problem), the app retries the same key with a short delay, instead of wasting the whole key pool.

10.2 Error Handling

The data layer distinguishes between different failure types and shows the user a clear, local-language message instead of a technical crash:

- Request timeout / no internet connection
- Malformed or unexpected server response
- Image blocked by AI safety filters
- All configured keys exhausted
- AI server temporarily busy

This makes the app resilient in the low-connectivity conditions common in rural field use — a key requirement given the target users are farmers, not developers.

11. Implementation Details

Key screens/flows built:

- **Splash screen** — initial load and Firebase setup
- **Login / Signup / Profile** — Firebase Authentication
- **Home navigation** — entry point to the detection feature
- **Plant Detection page** — camera/gallery image capture and preview
- **Analysis flow** — sends the image to the AI backend and shows a loading state
- **Result page** — shows health status, disease name (Bengali & English), severity, symptoms, and recommended medicine/dosage

The AI prompt sent to Gemini is written in Bengali and specifically asks for: disease status, disease name, severity, symptoms, recommended medicines with dosage, and care steps — matching the target users' language.

12. Results and Evaluation

12.1 Model Performance (YOLOv12n)

Metric	Value
Overall accuracy (mAP@50)	73.8%
Precision	0.711
Recall	0.732
Best-performing class	Early Blight (90.3% mAP)
Most challenging class	Bacterial Spot (50.5% mAP)
Inference latency	6.8 ms (Tesla T4)
Optimal confidence threshold	0.234 (peak F1 = 0.72)

Class-wise detection (confusion matrix, normalized)

Class	True Positive Rate
Leaf Miner	0.82
Bacterial Spot	0.55
Early Blight	0.91

The model correctly detects diseases roughly 3 out of 4 times overall. Early Blight is detected very reliably because of its distinct ring-shaped pattern. Bacterial Spot is harder because leaves in the dataset often contain many small, tightly packed spots (about 10 per leaf on average), which the model tends to confuse with background.

12.2 Comparative Analysis

Study	Architecture	Disease Classes	Performance	Deployment Focus
Tm et al. (2018)	LeNet CNN	Multiple	94–95% Accuracy	Limited computing
Agarwal et al. (2020)	Simplified CNN	Multiple	98.4% Accuracy	Lab/static dataset
Rajamohanam (2023)	YOLOv5	Multiple	93% Accuracy	Real-time field
Abulizi et al. (2025)	DM-YOLO (v9)	Multiple	95.1% mAP	Real-time precision

Wang & Liu (2025)	TomatoGuard (v10)	Multiple	94.23% mAP50	High speed (129 FPS)
Shen et al. (2025)	Optimized YOLOv8	Multiple	+2.2% Precision	Feature extraction
Our Research (2026)	YOLOv12n	Multiple	0.738 mAP50 (0.903 for Early Blight)	Edge/Tesla T4 (6.8 ms latency)

Compared to prior work, our model's overall accuracy is lower than several lab-only studies, but this is a fair trade-off: our dataset includes more crowded, real-world-style images, and the model (YOLOv12n-nano) is deliberately chosen for its very low latency so it can run on lightweight/edge devices rather than only in a research lab.

12.3 Mobile Application Testing

The application was verified through functional testing of the full user flow (image capture → analysis → result display) and targeted testing of failure conditions:

- Simulated API key expiry/rejection → confirmed automatic rotation to the next key
- Simulated server error → confirmed retry-with-backoff behavior
- Simulated no-internet condition → confirmed a clear, user-facing error message instead of a crash

Automated unit/widget test coverage is limited at this stage (default Flutter widget test only); expanding automated test coverage is listed under Future Work.

13. Working Schedule (Gantt Chart)

Task	Duration	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10
Problem Identification	1 week	✓									
Literature Review	1 week		✓								
Data Collection & Preprocessing	2 weeks			✓	✓						
Model Training	3 weeks					✓	✓	✓			
Testing and Validation	1 week								✓		
Mobile App Development	(ongoing, parallel)										
Report/Paper Organization	2 weeks									✓	✓

14. Limitations

- The YOLOv12n model's accuracy on Bacterial Spot (50.5% mAP) is noticeably lower than other classes due to crowded small lesions in the training images.
- The test set (20 images) is small; a larger test set would give a more reliable accuracy estimate.
- The mobile app currently uses a general-purpose cloud AI (Gemini) rather than the custom-trained YOLO model directly on-device; this depends on internet connectivity and third-party API availability.
- Automated software testing (unit/integration tests) for the mobile app is minimal at this stage.

15. Expected Outcome and Future Work

- Integrate the trained YOLOv12n model directly into the mobile app (on-device inference), removing dependency on external AI services and enabling offline use.
- Expand the dataset, especially for Bacterial Spot, to improve detection of small, crowded lesions.
- Explore drone-based field scanning: given the model's 6.8 ms inference speed, a drone could scan a whole field in real time and flag disease "hotspots" automatically.
- Add automated test coverage for the mobile application.
- Pilot the app with real farmers to collect feedback on usability and translation accuracy.

16. Conclusion

This project combines a research-grade YOLOv12n model for tomato leaf disease detection with a working, farmer-facing Flutter mobile application. The model achieves a 73.8% mAP@50 with very low inference latency, making it suitable for real-time and edge deployment, while the mobile app already delivers instant, Bengali-language diagnosis and treatment advice today using cloud AI, with a clear path to bring the custom model on-device. Together they demonstrate a complete pipeline from research to a real, usable agricultural tool.

17. References

1. Agarwal, M., Gupta, S. K., & Biswas, K. K. (2020). Development of efficient CNN model for tomato crop disease identification. *Sustainable Computing: Informatics and Systems*, 28, 100407.
2. Ahmed, S., Hasan, M. B., Ahmed, T., Sony, M. R. K., & Kabir, M. H. (2022). Less is more: Lighter and faster deep neural architecture for tomato leaf disease classification. *IEEE Access*, 10, 68868–68884.
3. Altalak, M., Uddin, M. A., Alajmi, A., & Rizg, A. (2022). A hybrid approach for the detection and classification of tomato leaf diseases. *Applied Sciences*, 12(16), 8182.
4. Attallah, O. (2023). Tomato leaf disease classification via compact convolutional neural networks with transfer learning and feature selection. *Horticulturae*, 9(2), 149.
5. Baek, E. T. (2025). Attention score-based multi-vision transformer technique for plant disease classification. *Sensors*, 25(1), 270.
6. Borugadda, P., Lakshmi, R., & Sahoo, S. (2023). Transfer learning VGG16 model for classification of tomato plant leaf diseases: A novel approach for multi-level dimensional reduction. *Pertanika Journal of Science & Technology*, 31(2).
7. Rajamohanam, R., & Latha, B. C. (2023). Optimized YOLOv5 for tomato leaf disease classification.
8. Abulizi, A., Ye, J., & Guo, W. (2024). DM-YOLO: An improved YOLOv9-based detector.
9. Tm, P., Pranathi, A., & Sai Ashritha, K. (2018). LeNet-based CNN for plant disease classification.
10. Shanthi, D. L., Vinutha, K., & Ashwini, N. (2024). Custom CNN architectures (AlexNet, VGGNet-16) for tomato disease classification.